



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Bank Transaction Processing System

Data Structures and Applications – IS233AI

Experiential Learning (Lab)

REPORT

Submitted by

Pavan Varma

USN 1RV24IS089

Nikhil Sharma

USN 1RV24IS081

Under the guidance of

Rekha B S

Assistant Professor, Department of Information Science and Engineering

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Information Science and Engineering

2025-2026



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled '**Bank Transaction Processing System**' is carried out by **Penumatcha Pavan Varma (1RV24IS089)**, **Nikhil Sharma (1RV24IS081)**, who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Information Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide

Guide Name

Signature of Head of the Department

Dr. G S Mamatha

External Viva

Name of Examiners

Signature with Date

1

2



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Penumatcha Pavan Varma (1RV24IS089)**, **Nikhil Sharma (1RV24IS081)**, students of Third semester B.E., department of Information Science Engineering, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled '**Bank Transaction Processing System**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Information Science and Engineering** during the academic year 2025-26

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering®, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1.Penumatcha Pavan Varma (1RV24IS089)

2.Nikhil Sharma (1RV24IS081)

ABSTRACT

THE PROPOSED PROJECTS OFFER AN OPTIMIZED BANKING TRANSACTION SYSTEM BY INTEGRATING TRADITIONAL DATA-STRUCTURE CONCEPTS WITH MINIMAL INTERFACES. UNLIKE MANY PROJECTS UNDERTAKEN IN THE PAST THAT UTILIZE DATABASES AND SERVER-SIDE FUNCTIONALITY, THIS PROPOSED ENDEAVOR EXPLORES: “WHAT’S THE CUTTING POINT AT WHICH YOU CAN PROTOTYPE BANKING WORKFLOWS WITH MEMORY DATA STRUCTURES AND SIMPLE PERSISTENCE?” THE INTENTIONS HERE ARE VERY SIMPLE—FAST ACCOUNT LOOKUPS WITH HASHING, ORDERLY HANDLING FOR QUEUED TRANSACTIONS, SAFE REVERSAL WITH STACKS, AND HISTORY VISIBILITY. THESE ARE PACKAGED INTO TWO INTERFACES—A CONSOLE INTERFACE (IN C) AND AN INTERNET-FACING (HTML-CSS-JAVASCRIPT) UI.

IT EMPLOYS PARALLEL IMPLEMENTATIONS. IT UTILIZES A HASH MAP FOR ACCOUNTS, A QUEUE FOR PENDING TRANSACTIONS, A STACK FOR ROLL-BACKS, AND A LINKED LIST FOR THE HISTORY OF TRANSACTIONS. IT EMPLOYS FILE PERSISTENCE FOR ACCOUNTS AS WELL AS THE HISTORY OF TRANSACTIONS. IT EMPLOYS A HASH MAP IN JAVASCRIPT FOR ACCOUNTS, A QUEUE FOR PENDING TRANSACTIONS, A STACK FOR ROLL-BACKS, AND A LINKED LIST IN LOCALSTORAGE FOR THE HISTORY OF TRANSACTIONS. IT EMPLOYS CHART.JS FOR THE VISUALIZATION OF THE RESULTS. IT REQUIRES NO EXTERNAL HARDWARE, AS IT RUNS ON ALL STANDARD DESKTOP COMPUTERS, ALONG WITH A TYPICAL C SOFTWARE DEVELOPMENT TOOLKIT AND A BROWSER.

THE RESULTS REVEAL THAT ACCESSING ACCOUNTS TAKES CONSTANT TIME, PENDING TRANSACTIONS ARE PROCESSED FOLLOWING THE FIFO RULE, WHILE THE ROLLBACKS ENSURE AN LIFO PATTERN, ENABLING IMMEDIATE UPDATES FOR BOTH IMPLEMENTATIONS TO ENSURE CORRECT HISTORY. IN THE BROWSER-BASED SYSTEM, THE DASHBOARD PROVIDES IMMEDIATE REFRESHES FOR METRICS SUCH AS TOTAL BALANCE AND PENDING TRANSACTIONS, WHILE IN THE C LANGUAGE SYSTEM, FEEDBACK UPON INTERACTION WITH THE CONSOLE INDICATES SUCCESS IN CHANGING THE BALANCE AS WELL AS PERFORMING THE NECESSARY ROLLBACKS. IN TERMS OF FUNCTIONALITY, THIS PROJECT PROVIDES, WITH LESS OVERHEAD, THE SAME FUNCTIONAL CAPABILITY AS PROPOSED WITHIN THE LITERATURE REVIEWS BASED ON FULL DATABASES, BUT WITH REDUCED SCALABILITY AND CONCURRENCY.

CONCLUSION: THIS DESIGN PROVES THAT A BANKING CORE PATTERN OF BEHAVIOR CAN BE PROPERLY REPRESENTED WITH A SMALL IN-MEMORY DATA REPRESENTATION AND PERSISTENCE LAYER. AREAS FOR FUTURE STUDY AND EXPANSION MIGHT INCLUDE SERVER-BASED STORAGE FOR MULTI-USER CAPABILITY, VALIDATION AND CONCURRENCY CONTROLS, ANALYSIS CAPABILITIES, AND INTEGRATION TESTS THAT COMPARE THE TWO ENVIRONMENTS.

TABLE OF CONTENTS

Abstract	Page no
Chapter 1: Introduction.....	1
• <i>Overview of the problem statement or case study,</i>	
• <i>its significance and the objectives of the proposed solution.</i>	
Chapter 2: Solution Design.....	2-4
• <i>Detailed design of the proposed solution which should include</i>	
<i>System Design/ Architecture representing the proposed system.</i>	
• <i>Selection and justification of appropriate data structures and operations used.</i>	
• <i>Flow Chart</i>	
Chapter 3: Implementation Details.....	5-6
• <i>Description of the implementation approach.</i>	
• <i>Use of coding best practices including modularity, readability,</i>	
<i>and maintainability.</i>	
Chapter 4: Results and Discussion.....	7-10
• <i>Execution outcomes and validation of algorithm correctness.</i>	
• <i>Handling of special or edge cases.</i>	
• <i>Comparative assessment of alternative algorithms.</i>	
• <i>Discussion of potential improvements or refinements.</i>	
Chapter 5: Conclusion and Future Scope.....	11-12
• <i>Summary of key findings and overall effectiveness of the solution.</i>	
• <i>Real-world applicability and relevance.</i>	
• <i>Additional innovation (if any): Novel approaches, enhanced optimizations,</i>	
<i>or industry/research applicability.</i>	
References	13

Introduction

Modern banking hinges on the ability to execute and audit transactions with consistency, speed, and reversibility. This case study examines how to capture those core behaviors—account creation, deposits, withdrawals, transfers, queued processing, rollbacks, and audit history—in a deliberately lightweight architecture. Instead of relying on heavyweight databases or service meshes, the project uses in-memory data structures and minimal persistence to prove that essential financial flows can be prototyped, tested, and demonstrated with clarity and low overhead. This approach is significant for early-stage validation, training, and algorithm exploration, where transparency and ease of iteration matter as much as raw scale.

The proposed solution sets out to meet three objectives. First, ensure responsive account access by employing a hash map keyed by account identifiers, so lookups remain effectively constant time even as accounts grow. Second, enforce disciplined transaction handling through a queue for pending items (FIFO to preserve arrival order) and a stack for rollbacks (LIFO to safely undo the most recent action), reflecting real operational needs for orderly processing and controlled recovery. Third, maintain an auditable, chronological transaction history to support accountability, reconciliation, and analytics. To illustrate portability, these objectives are delivered in two parallel implementations: a C console application that persists to simple text files, and a browser-based dashboard written in HTML/CSS/JavaScript that persists to localStorage, offers form-driven interactions, and uses Chart.js for immediate visual feedback. Together, they demonstrate that a consistent domain model can span multiple environments while keeping the design transparent and easy to evolve.

Solution Design

2.1 System Design and Architecture of the Proposed System

The proposed system is designed as a modular banking transaction simulator that models essential banking operations such as account management, transaction processing, rollback, and audit logging. The system is implemented in two environments: a C-based console application and a browser-based web application. Despite differences in platform and user interface, both implementations follow the same conceptual system architecture and operational flow. This unified design ensures consistency in behavior and allows the system to demonstrate how identical problem domains can be addressed across different execution environments.

At a high level, the system architecture follows a layered design approach. The user interaction layer is responsible for capturing user inputs and presenting system outputs. In the console-based version, this layer is implemented through a menu-driven interface that allows users to select operations such as adding accounts, processing transactions, and performing rollbacks. In the web-based version, the same functionality is exposed through interactive forms, buttons, and dynamically updated views. Although the interaction mechanisms differ, both interfaces invoke the same logical operations in the underlying application layer.

The application logic layer forms the core of the system and coordinates all banking activities. This layer handles validation of inputs, enforcement of business rules, and orchestration of transaction flow. When a user initiates an operation, such as adding a transaction or processing pending requests, the application logic layer determines the correct sequence of actions to be executed. This includes verifying the existence of accounts, checking balance constraints, ensuring correct transaction ordering, and triggering updates to account balances, rollback structures, and history logs. By centralizing these responsibilities, the design ensures that business rules are consistently applied across all operations.

The data handling layer is responsible for organizing and maintaining runtime data using well-defined abstract data types. Account information, pending transactions, rollback data, and transaction history are all stored in memory using appropriate data structures chosen to reflect real-world banking behavior. This layer abstracts the underlying storage mechanisms from the application logic, allowing the system to operate independently of the user interface and persistence method. Such separation improves clarity and maintainability of the design.

The persistence layer ensures that key system state persists between runs. In the C implementation, this is done via file-based persistence: account details and transaction history are written and read to/from text files. In the web implementation, persistence is based on browser-based local storage, enabling data to survive reloads without a backend server. The

persistence mechanisms have been designed simple and light on purpose, enhancing the educative nature of the system while still showing state preservation concepts.

Overall, the system architecture is such that it resembles real-world transaction processing pipelines but in a very simplified form. The system separates concerns into interaction, logic, data management, and persistence; therefore, clarity, correctness, extensibility, and ease of understanding and implementation are achieved.

2.2 Selection and Justification of Data Structures and Operations

The selection of data structures in the proposed system is guided by the necessity for the effective modeling of banking operations at the same time as striving for efficiency and logical correctness. Each core banking component maps to a data structure whose properties naturally align with the required behavior.

An account record is stored in a hash-based data structure, indexed by a unique account identifier. This data structure allows for near-constant time complexities for operations such as lookups, insertions, and updates. This is important, as banking systems typically entail considerable frequency in account access. The hashing employed here allows balance updates, the validation of accounts, or transfers without having to scan large collections of accounts. It also makes checking for duplicate accounts and missing account references simpler.

Pending transactions are kept in a queue data structure. Transactions join this queue in the sequence they are produced and are executed one after another. This first-in-first-out behavior closely matches the way a real banking system works, where transactions are normally processed in order of arrival to preserve fairness and consistency. Queue operations like enqueue and dequeue are simple and effective, leaving no chance of any transaction being missed or out-of-order during processing.

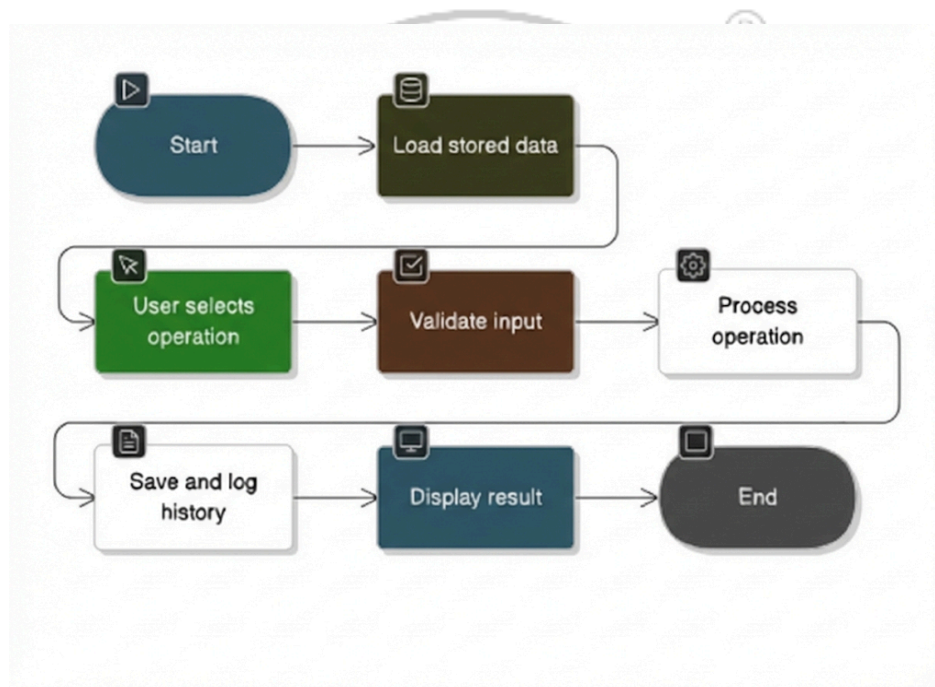
The rollback mechanism is implemented by using a stack data structure. A stack follows the last-in-first-out principle, making it ideal for undo operations. As a transaction is processed, it is pushed onto the rollback stack. In the event of a rollback, the most recently processed transaction is popped off the stack and reversed. This ensures correctness because only the most recent operation that changed the state is undone, thereby avoiding inconsistencies that may arise from reversing older transactions out of order.

A singly linked list maintains the transaction history. Every processed transaction gets added to the end, thus giving a clear, time-ordered audit trail. The linked list allows insertion of new entries efficiently without resizing or shifting elements as an array-based approach would require. Because the list is append-only, it mimics real banking ledgers, where records are

immutable and retained for audits and compliance. This structure also enables simple traversal sequentially for the display or filtering of past transactions.

Correctness is preserved by specific sequences of operations on these data structures. An account needs to be retrieved, followed by the update of its balance. To process a transaction, take it from pending, update the account, push it onto the rollback stack, and append it to history. The rollback entails a pop from the stack, with the application of the reverse balance adjustment. At every step in the state transitions, there are embedded checks against invalid states, such as negative balances, an empty rollback stack, or missing accounts.

In other words, the abstract notions of banking map naturally and efficiently to concrete computational models via the choice of data structures and their operations. This approach serves to simplify the implementation while elucidating how core data structures underpin real-world transaction processing systems.



Implementation Details

3.1 Description of the Implementation Approach

The implementation of the Banking Transaction Simulator follows a structured and incremental approach, ensuring that correctness and clarity are maintained throughout development. The system was implemented in two parallel environments: a C-based console application and a browser-based web application. While the programming languages and execution platforms differ, both implementations adhere to the same logical design and operational flow defined in the solution design phase. This approach allows the system to demonstrate platform-independent problem modeling while highlighting the strengths of each implementation environment.

Development began with the definition of core entities such as accounts and transactions, followed by the implementation of fundamental data structures required to support system behavior. In the C implementation, memory was dynamically allocated for data structures such as linked lists, stacks, and queues, with explicit management of allocation and deallocation to prevent memory leaks. File-based persistence was then integrated to ensure that account data and transaction history could be saved and restored across executions. Each functional requirement, such as adding accounts, processing transactions, and performing rollbacks, was implemented as a separate function to maintain clarity and logical separation.

In the web-based implementation, the same core logic was translated into JavaScript using object-oriented constructs. Custom classes were created to represent data structures such as hash maps, queues, stacks, and linked lists. The user interface was implemented using HTML and CSS, while JavaScript handled all application logic and state management. Browser local storage was used to persist data, enabling the system to restore state upon page reload without requiring a backend server. Interactive elements such as dashboards and transaction views were updated dynamically to provide immediate feedback to the user.

In both implementations, operations were designed to follow a consistent transaction lifecycle. User inputs are first validated, followed by execution of the requested operation, update of relevant data structures, and finally persistence of the updated state. This consistent flow ensures predictable behavior and simplifies debugging and extension of the system.

3.2 Coding Best Practices: Modularity, Readability, and Maintainability

Modularity was a primary consideration in the implementation of the system. Each major operation, such as account creation, transaction enqueueing, transaction processing, rollback handling, and data persistence, was implemented as a separate logical unit. In the C implementation, this was achieved through well-defined functions, each performing a single

responsibility. In the JavaScript implementation, modularity was achieved using classes and clearly separated functions for user interface handling and business logic. This modular design makes the system easier to understand, test, and extend.

Readability was ensured through consistent naming conventions, clear function and variable names, and logical grouping of related operations. Descriptive identifiers were used to reflect the purpose of variables and functions, reducing the need for excessive comments. Code blocks were structured in a clear and predictable manner, and indentation was consistently applied to enhance visual clarity. Input validation and error handling were explicitly implemented to prevent undefined behavior and to make execution paths easier to follow.

Maintainability was addressed by minimizing dependencies between components and avoiding duplication of logic. Common validation routines and data manipulation operations were reused wherever possible. In the C implementation, careful attention was paid to memory management, with dynamic allocations paired with appropriate deallocation to avoid memory leaks. In the JavaScript implementation, application state was centralized and synchronized with persistent storage to avoid inconsistencies. Clear separation between data handling and user interface logic ensures that changes to one part of the system have minimal impact on others.

Additionally, defensive programming practices were adopted to improve system robustness. All critical operations include checks for invalid input, missing data, or invalid system states, such as empty queues or stacks. These safeguards prevent system crashes and make the system more resilient to incorrect usage.

Overall, the implementation follows established software engineering best practices, resulting in a system that is easy to read, modify, and extend. The structured approach ensures that new features, such as additional transaction types or alternative persistence mechanisms, can be incorporated with minimal changes to the existing codebase.

Results and Discussion

4.1 Execution Outcomes and Validation of Algorithm Correctness

The execution of the Banking Transaction Simulator demonstrates that the system correctly implements the intended banking operations across both the C console application and the web-based implementation. During execution, the system successfully supports account creation, transaction enqueueing, transaction processing, rollback operations, fund transfers, and transaction history maintenance. The consistency of results across multiple runs validates the correctness of the underlying algorithms and data structure interactions.

Algorithm correctness is primarily validated through the predictable behavior of transactions and balances. When a deposit transaction is processed, the corresponding account balance increases by the specified amount, and when a withdrawal transaction is processed, the balance decreases accordingly, provided sufficient funds are available. The transaction processing pipeline consistently follows the defined order: pending transactions are processed in FIFO order, processed transactions are recorded in history, and rollback information is maintained in LIFO order. The correct synchronization of these structures confirms that the algorithms behave as expected.

Rollback functionality further validates correctness. When a rollback operation is performed, the most recently processed transaction is reversed accurately, restoring the account balance to its previous state. This confirms that the stack-based rollback mechanism correctly preserves transaction order and reversal semantics. The transaction history remains intact, providing a complete audit trail, which reinforces the integrity of the execution logic.

Persistence mechanisms also contribute to correctness validation. Data stored in files (for the C version) or browser local storage (for the web version) is correctly reloaded on subsequent executions, ensuring continuity of account balances and transaction records. This confirms that serialization and deserialization of system state are implemented correctly.

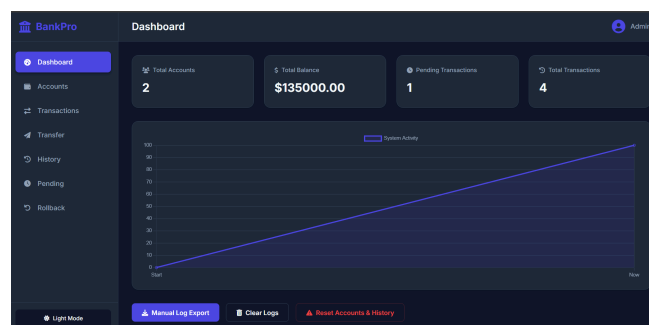


Fig 4.1 Dashboard interface of the BankPro system showing total accounts, balance summary, transaction status, and system activity graph.

4.2 Handling of Special and Edge Cases

The system explicitly handles several special and edge cases to ensure robustness and prevent invalid states. One important edge case is the attempt to perform operations on non-existent accounts. In such cases, the system detects the error during validation and prevents further processing, thereby avoiding undefined behavior. Similarly, duplicate account creation requests are rejected to maintain the uniqueness of account identifiers.

Another critical edge case involves insufficient funds during withdrawal or transfer operations. The system checks account balances before applying deductions and rejects transactions that would result in negative balances. This safeguard ensures financial consistency and prevents invalid account states. Empty data structures are also handled gracefully; attempts to process transactions from an empty pending queue or perform rollback operations on an empty stack result in informative error messages rather than system failures.

Invalid inputs, such as negative transaction amounts or incorrect transaction types, are rejected during the validation phase. This defensive programming approach ensures that only valid data enters the system, reducing the likelihood of runtime errors. In the web-based implementation, additional checks are performed at the user interface level to provide immediate feedback, further enhancing system reliability.

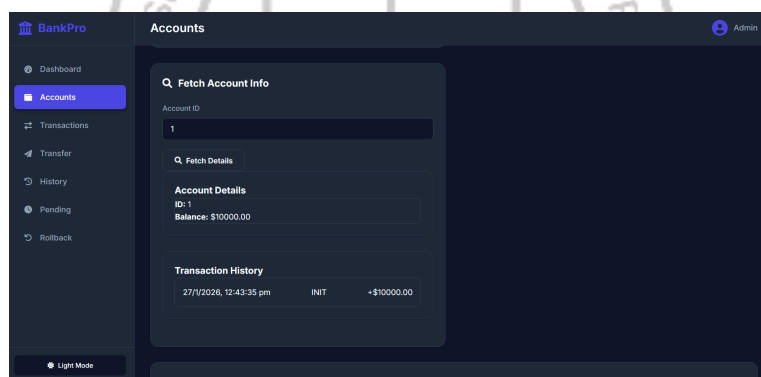


Fig 4.2 User interface of the BankPro Accounts module showing account information retrieval and transaction history.

4.3 Comparative Assessment of Alternative Algorithms

Alternative algorithmic choices were considered during the design phase. For account storage, a linear list or array-based structure could have been used instead of a hash map. However, such approaches would result in linear-time lookups, which become inefficient as the number of accounts increases. The hash map provides significantly better performance and aligns with real-world banking requirements.

For transaction processing, an alternative approach could involve processing transactions immediately upon entry rather than placing them in a queue. While this simplifies

implementation, it eliminates the ability to control processing order and simulate real-world transaction pipelines. The queue-based approach was chosen because it accurately models sequential transaction handling and allows for batch or deferred processing.

Rollback functionality could alternatively be implemented by recomputing balances from transaction history rather than using a stack. While this method may reduce memory usage, it increases computational complexity and makes rollback operations slower. The stack-based approach offers constant-time rollback and clearer undo semantics.

For transaction history, an array-based list could have been used instead of a linked list. However, arrays require resizing and shifting of elements, which can be inefficient for append-heavy workloads. The linked list approach allows efficient append operations and naturally supports an immutable audit trail.

Overall, the selected algorithms strike a balance between simplicity, correctness, and efficiency, making them well-suited for the scope of the project.

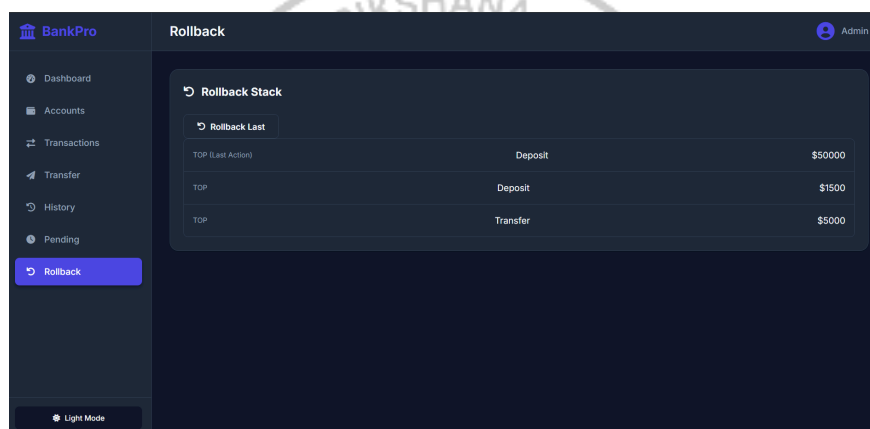


Fig 4.3 Rollback interface of the BankPro system illustrating stack-based transaction recovery and last-action reversal.

4.4 Discussion of Potential Improvements and Refinements

Although the system achieves its intended objectives, several enhancements could be made to improve functionality and scalability. One potential improvement is the introduction of concurrency control mechanisms to support multiple simultaneous users or transactions. This would require synchronization techniques such as locks or transaction isolation strategies.

Another refinement involves replacing file-based or local storage persistence with a database-backed solution. Using a relational or NoSQL database would enable more reliable storage, advanced querying, and support for larger datasets. Authentication and authorization mechanisms could also be added to restrict access and enhance security.

From an algorithmic perspective, the system could be extended to support multi-level transaction states, such as pending, committed, and rolled-back, to more closely resemble real-world banking systems. Automated testing and logging frameworks could further improve reliability and maintainability. Additionally, modularization could be enhanced by separating the data structure layer into reusable libraries.

Despite these possible extensions, the current system effectively demonstrates how fundamental data structures and algorithms can be combined to model real-world banking operations in a clear and educational manner.

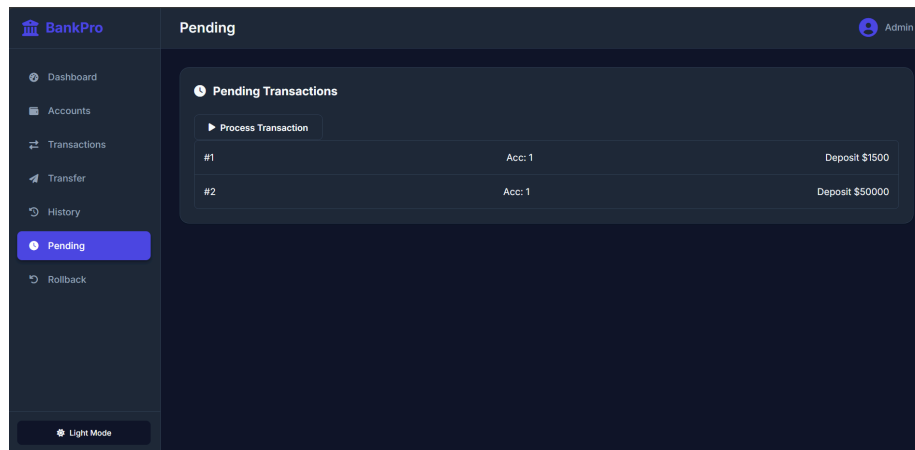


Fig 4.4 Pending transactions interface showing queued deposit requests awaiting processing in the BankPro system.

Conclusion and Future Scope

5.1 Conclusion and Summary of Key Findings

This project successfully designed and implemented a Banking Transaction Simulator that models essential banking operations using fundamental data structures and algorithms. The solution demonstrates how abstract concepts such as accounts, transactions, rollback mechanisms, and audit trails can be translated into a working system with clear operational flow. By implementing the same logical design in both a C-based console application and a browser-based web application, the project establishes the effectiveness of platform-independent system modeling.

The results show that the selected data structures—hash maps for account storage, queues for pending transactions, stacks for rollback operations, and linked lists for transaction history—work together cohesively to maintain correctness, efficiency, and clarity. The system consistently preserves transaction order, prevents invalid operations such as overdrafts and duplicate accounts, and supports accurate rollback of recent transactions. Persistence mechanisms further ensure continuity of data across executions, reinforcing the reliability of the solution.

Overall, the system achieves its intended objective of demonstrating transaction processing concepts in a transparent and educational manner. The modular architecture, clear separation of concerns, and defensive handling of edge cases contribute to the overall effectiveness and robustness of the solution.

5.2 Real-World Applicability and Relevance

Although the system is designed as a simplified simulator, it closely reflects real-world banking transaction workflows. Concepts such as pending transaction queues, audit logs, rollback mechanisms, and balance validation are foundational elements of actual financial systems. The project therefore provides strong conceptual relevance to real-world applications in banking, financial technology, and transaction-based systems.

The simulator is particularly useful in educational contexts, where it can be used to teach data structures, algorithmic thinking, and system design principles. It also serves as a prototype model for understanding transaction pipelines, error recovery, and state persistence. While the system does not implement advanced features such as concurrency control, authentication, or distributed processing, its core logic mirrors the fundamental building blocks used in real banking software.

The web-based implementation further enhances applicability by demonstrating how client-side applications can manage structured data, maintain state, and provide interactive feedback without

relying on a backend server. This aligns well with modern trends in rapid prototyping and frontend-driven application development.

5.3 Additional Innovation and Future Scope

The primary innovation of this project lies in its unified modeling approach across two different platforms while maintaining identical domain logic. Implementing the same banking workflow using low-level procedural programming in C and high-level object-oriented programming in JavaScript highlights the adaptability of the design and reinforces core system design principles. The explicit use of classic data structures to represent real-world banking behavior provides educational clarity and strengthens conceptual understanding.

Several opportunities exist for future enhancement and refinement. The system can be extended to support concurrent users and simultaneous transaction processing through the introduction of synchronization mechanisms or transactional locking. Persistence can be improved by integrating a database-backed storage system, enabling scalability and long-term data management. Security features such as authentication, authorization, and encryption could be added to make the system more suitable for real-world deployment scenarios.

From a research and industry perspective, the simulator can serve as a foundation for exploring advanced topics such as event sourcing, transaction isolation levels, distributed ledger systems, and fault-tolerant transaction processing. Additional optimizations, such as batch transaction processing and performance monitoring, could further enhance system efficiency. Automated testing frameworks and logging mechanisms could also be incorporated to improve reliability and maintainability.

In conclusion, while the current system is intentionally simple, it provides a strong and extensible foundation. With further development, it has the potential to evolve into a more sophisticated transactional system suitable for both academic research and industry-oriented experimentation.

References:

1. **[1] Mohan et al. (2024)**, Mohan et al. presented the ARIES recovery mechanism based on Write-Ahead Logging (WAL) for database transaction recovery. Their work focuses on log sequence numbers and page-level state management, making it suitable for large-scale DBMS environments but overly complex for small academic or educational systems.
2. **[2] Silberschatz et al. (2024)**, Silberschatz et al. discussed the theoretical foundations of queues and stacks in scheduling and undo operations within operating systems and transaction systems. While conceptually strong, the work lacks simplified, practical implementations tailored for student-level banking or transactional simulations.
3. **[3] Elmasri and Navathe (2024)** , Elmasri and Navathe explored transaction logs and history maintenance in database systems, emphasizing DBMS-centric architectures. Their approach relies heavily on database engines, making it less suitable for lightweight or file-based transaction processing systems.
4. **[4] Krishna Mula (2025)**, Krishna Mula focused on real-time transaction processing systems with an emphasis on performance and latency optimization. Although effective for high-throughput systems, the work does not adequately address rollback modeling for small or educational transaction systems.
5. **[5] Cormen et al. (2024)**, Cormen et al. analyzed hash tables as efficient data structures for fast data access. While their work demonstrates excellent lookup performance, it does not address transaction ordering, scheduling, or recovery mechanisms required in banking systems
6. **[6] Rule-Based Transaction Systems (2024)**, Rule-based transaction systems employ fixed rule sets for fraud detection and transaction validation. These systems are rigid in nature, offering limited adaptability and basic alerting capabilities without support for extensible or dynamic transaction logic.